

SODA: An Adaptive Bitrate Controller for Consistent High-Quality Video Streaming

Tianyu Chen
University of Massachusetts Amherst
Amherst, MA, USA
tianyuchen@umass.edu

Yiheng Lin
California Institute of Technology
Pasadena, CA, USA
yihengl@caltech.edu

Nicolas Christianson
California Institute of Technology
Pasadena, CA, USA
nchristianson@caltech.edu

Zahaib Akhtar
Amazon Prime Video / NCSU
Sunnyvale, CA, USA
akhtz@amazon.com

Sharath Dharmaji
Amazon Prime Video
Sunnyvale, CA, USA
sharatdr@amazon.com

Mohammad Hajiesmaili
University of Massachusetts Amherst
Amherst, MA, USA
hajiesmaili@cs.umass.edu

Adam Wierman
California Institute of Technology
Pasadena, CA, USA
adamw@caltech.edu

Ramesh K. Sitaraman
University of Massachusetts Amherst
Amherst, MA, USA
ramesh@cs.umass.edu

ABSTRACT

The primary objective of adaptive bitrate (ABR) streaming is to enhance users' quality of experience (QoE) by dynamically adjusting the video bitrate in response to changing network conditions. However, users often find frequent bitrate switching frustrating due to the resulting inconsistency in visual quality over time, especially during live streaming when buffer lengths are short. In this paper, we propose a practical smoothness optimized dynamic adaptive (SODA) controller that specifically addresses this problem while remaining deployable. SODA is backed by theoretical guarantees and has shown superior performance in empirical evaluations. Specifically, our numerical simulations show a 9.55% to 27.8% QoE improvement and our prototype evaluation shows a 30.4% QoE improvement compared to the state-of-the-art baselines. In order to be widely deployable, SODA performs bitrate horizon planning in polynomial time compared to brute force approaches that suffer from exponential complexity. To demonstrate its real-world practicality, we deployed SODA on a wide range of devices within the production network of Amazon Prime Video. Production experiments show that SODA reduced bitrate switching by up to 88.8% and increased average stream viewing duration by up to 5.91% compared to a fine-tuned production baseline.

CCS CONCEPTS

• Information systems → Multimedia streaming; • Theory of computation → Online algorithms; Theory and algorithms for application domains.

KEYWORDS

Adaptive bitrate streaming, Smoothed online convex optimization

ACM Reference Format:

Tianyu Chen, Yiheng Lin, Nicolas Christianson, Zahaib Akhtar, Sharath Dharmaji, Mohammad Hajiesmaili, Adam Wierman, and Ramesh K. Sitaraman. 2024. SODA: An Adaptive Bitrate Controller for Consistent High-Quality Video Streaming. In *ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*, August 4–8, 2024, Sydney, NSW, Australia. ACM, New York, NY, USA, 32 pages. <https://doi.org/10.1145/3651890.3672260>

1 INTRODUCTION

With the growth of online video streaming, users nowadays stream videos from a highly diverse set of devices, including laptops, mobile devices, smart TVs, set-top boxes, game consoles, etc. These devices span a wide spectrum of hardware capabilities and connect to the Internet in a multitude of ways, e.g., wireless, cellular, cable, etc. To ensure a high quality of experience (QoE) across all devices, video providers utilize adaptive bitrate (ABR) streaming that tailors video delivery to specific devices and network conditions.

The goal of ABR streaming is to deliver a video at the highest sustainable quality over time-varying network conditions. To achieve this, a video source is encoded at different bitrates corresponding to different resolutions, e.g., 720p, 1080p, 1440p, etc. Each encoding is in turn temporally partitioned into a sequence of *segments*, e.g., 2 seconds of video content. An ABR controller inside a user's video player then selects a suitable bitrate for each segment. Finally, downloaded segments are stored in a buffer, till they are rendered.

Past studies have shown that a user's QoE is maximized by delivering the video at the highest possible quality with minimal rebuffering and bitrate switching. It has been shown that a 1% increase in rebuffering time is correlated with a 3-minute reduction in the viewing duration [7] and frequent bitrate switching is strongly correlated with a user abandoning the session [21]. Going beyond correlational studies, the significant *causal* impact of rebuffering and other QoE performance metrics on key measures of user behavior was first established in [9]. However, jointly optimizing all three key components of QoE, i.e., video quality, rebuffering and bitrate



This work is licensed under a Creative Commons Attribution International 4.0 License.
ACM SIGCOMM '24, August 4–8, 2024, Sydney, NSW, Australia
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0614-1/24/08
<https://doi.org/10.1145/3651890.3672260>

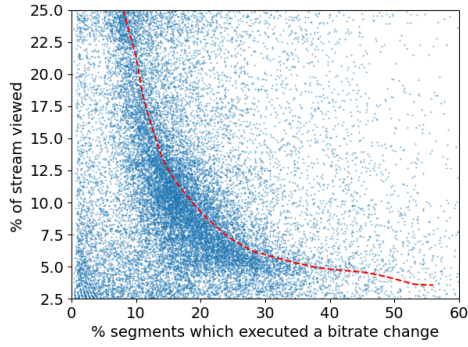


Figure 1: Video stream duration is negatively correlated with bitrate switching rate. Users watch < 10% of the stream when bitrate switching rate is > 20%.

switching, is non-trivial as they are locked in a three-way trade-off. An ideal ABR controller seeks to push the trade-off boundary and optimize all three QoE components *simultaneously*.

Between on-demand and live streaming, the latter is more challenging as the player buffer is restricted to 10 - 20 seconds (to remain close to actual live action), which is in contrast to 60 - 180 seconds of buffer in on-demand streaming. Consequently, live streaming has higher susceptibility to rebuffering and bitrate switching. To understand the impact of bitrate switching, Figure 1 shows the relationship between the viewing percentage of a stream and bitrate switching rate for a sports event on a large-scale video streaming provider. To minimize potential confounders such as rebuffering and low quality, the plot is focused on short-lived sessions (< 25% of stream viewed) with at least HD quality and no rebuffering. The line of best fit shows that users watch < 10% of the stream when bitrate switching rate is > 20%. While our proposed ABR controller works for both on-demand and live streaming, our evaluations use live streams that represents a more challenging use case.

Our Contributions. We propose a novel smoothness optimized dynamic adaptive (SODA) controller that provides theoretical QoE guarantees while exhibiting superior empirical performance in simulation, prototype, and production experiments. We make the following specific contributions:

- 1) **Theoretical Foundations of ABR Controller Design.** SODA is the first ABR controller to *provably* optimize *all three* key components of QoE, namely, video quality, rebuffering and bitrate switching. Unlike prior work such as BOLA [36, 44] that use Lyapunov methods to optimize the first two components, we use a new framework based on recent advances in smoothed online convex optimization (SOCO) [13, 25, 26, 33–35, 43] to simultaneously optimize all three QoE components. To enable the application of SOCO, we model the rebuffering minimization requirement in a novel fashion using the notion of buffer stability. We prove that SODA is near-optimal and achieves QoE within a small factor of the offline optimal QoE (Theorem 4.1).
- 2) **Better QoE Across Empirical Evaluations.** We evaluated SODA in three settings: numerical simulations, prototype evaluation, and production deployment within Amazon Prime Video serving actual users. Our numerical simulations show a 9.55% to

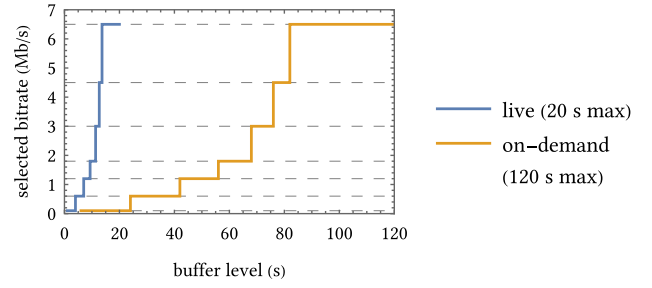


Figure 2: BOLA's [44] decision boundaries are spaced out for on-demand streaming, but tiny fluctuations in buffer level can cause bitrate switching for live streaming.

27.8% QoE improvement and our prototype evaluation shows a 30.4% QoE improvement compared to the state-of-the-art baselines. Production live streaming experiments in Amazon Prime Video show that SODA reduced bitrate switching by a significant up to 88.8% and increased average stream viewing duration by up to 5.91% (> 5 minutes longer sessions) compared to a fine-tuned production baseline. See Table 1 for a summary of our key findings about SODA as compared to baseline ABR controllers.

- 3) **Robustness Against Throughput Prediction Errors.** Most ABR controllers rely on and are sensitive to predictions of the future network throughput. Our SOCO framework allows us to design robust ABR controllers that are *provably* robust against prediction errors. Specifically, we show that SODA has the *exponentially decaying perturbation property* [49, 55, 56], i.e., the future impact of prediction errors decay rapidly over time. A key to our proof methodology is that we shifted from the conventional segment-based ABR formulation and adopted a novel *time-based* perspective.
- 4) **Efficient Implementation for Production Deployment.** ABR controllers deployed in the field need to work on a wide range of client devices, including low-end ones with limited computational resources. Many ABR controllers proposed in the research literature do not meet the efficiency bar for a production deployment and are never implemented in practice. We maximized SODA's runtime and deployment practicality by devising a computationally efficient method to search for near-optimal bitrate decisions, which reduced the runtime complexity from exponential to polynomial, e.g., about 200 iterations max in practice. In addition, we made SODA robust against throughput prediction errors by design, thus eliminating the need for sophisticated computationally-intensive throughput predictors.

This work does not raise any ethical issues.

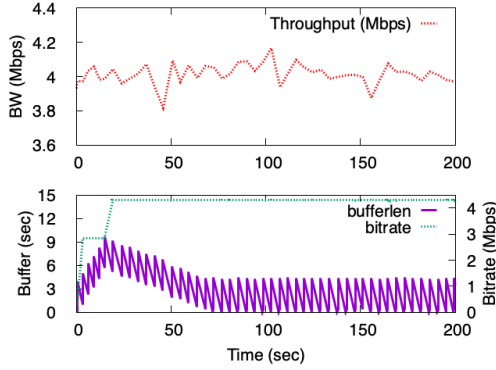
2 DESIGN GAPS, OPPORTUNITIES, AND REQUIREMENTS

Design Gaps. Live streaming poses the additional constraint of near real-time delivery which makes bitrate adaptation more challenging than that in on-demand streaming. Figure 2 shows the bitrate selection function of BOLA [36, 40, 44], an ABR controller that is widely deployed by video providers and is part of the reference MPEG-DASH video player [64]. Notice that for on-demand

Table 1: A qualitative summary of our key evaluation findings about SODA as compared to baseline ABR controllers.

Controller	Theory ^a	Video Quality	Rebuffering Time	Switching Rate	Deployability
SODA	Q + R + S	high	short	ultra low	high
HYB [24]	none	high	medium	high	high
BOLA [44]	Q + R	high	short	high	high
Dynamic [36]	Q + R	high	short	medium	high
MPC [17]	none	high	long	low	low
Fugu [46]	none	high	medium	low	low
CausalSimRL [60]	none	high	short	high	low

^aQ, R, S stand for theoretical guarantees for quality, rebuffering, and switching respectively.

**Figure 3: A RobustMPC session where the controller intentionally rebuffers instead of lowering the bitrate.**

streaming, a longer buffer of 120 seconds ensures that bitrate jumps are spaced well apart (up to 20 seconds), however, for live streaming with a buffer of 20 seconds, bitrates fluctuate with small deviations of 1 - 3 seconds in the buffer size. This can cause bitrates to switch frequently. While controllers such as MPC [17] and Pensieve [22] offer respite by explicitly penalizing bitrate switching, these controllers suffer from shortcomings of their own:

- Model predictive controllers are hard to deploy at scale because they need to solve a non-linear integer programming problem over a prediction horizon of K segments, e.g., $K = 5$, which is so computationally expensive that it is quicker to download a video segment than to obtain a bitrate decision [17, 19]. Workarounds such as pre-computed lookup tables [17] are impractical for live streaming where the video is not available a priori. In a similar vein, learning-based controllers such as Pensieve [22] work optimally when trained specifically for a given set of bitrates, segment duration, network conditions, etc. Given in-the-wild diversity and its evolving nature, ensuring this specificity imposes a significant operational overhead. Furthermore, even if specifically trained, achieving performance guarantees with these learning-based controllers is shown to be challenging [24].
- Existing ABR controllers naively reduce bitrate switching at the expense of low video quality or more rebuffering. To demonstrate this, Figure 3 shows a RobustMPC session with the exact setup

used by [17, 22]. Notice that beyond 70 seconds, RobustMPC repeatedly rebuffers but continues to download the highest bitrate (Figure 3 bottom plot), resulting in 29 rebuffering events over 200 seconds. Strikingly, this behavior is in fact the optimal behavior under RobustMPC's objective function which tolerates rebuffering to prevent bitrate switches. On the surface, this suggests higher rebuffering penalty in the objective function, however, higher penalties only reduce the duration of these *tolerable* rebuffers but do not eliminate them. Indeed, past work has empirically shown that even a 20× buffering penalty has marginal impact [24].

- Variance in network conditions or throughput prediction errors are not well tolerated by existing controllers. Past works have shown that RobustMPC incurs 26% more rebuffering events unless paired with a sophisticated throughput predictor [46, 50]. Similarly, learning-based controllers like Pensieve tend to degrade in performance when trained for realistic network conditions encountered in the wild [24]. In practice, accurate throughput predictions are hard because of several factors, including (i) device and OS level inefficiencies [18], (ii) stop-start nature of video requests which do not interact well with TCP [8, 18], and (iii) volatile network conditions typical in production networks [1, 5, 45]. To make matters worse, sophisticated throughput predictors are themselves not necessarily accurate [16] and are challenging to deploy due to device level bottlenecks [58]. Therefore, in-the-wild performance of these controllers remains questionable.

Opportunities. Outside of the video streaming literature, the interaction of learning and control has blossomed in recent years, leading to new and exciting approaches to controller designs [39, 47, 49, 52, 54]. However, these new approaches have not yet been applied and evaluated in the context of video streaming where classical model predictive and proportional–integral–derivative control have remained the focus, e.g., [17, 23]. In particular, the area of smoothed online convex optimization (SOCO) has seen multiple breakthroughs in recent years [13, 25, 33, 35], including the development of connections to model predictive controllers [26, 47, 49, 52, 56]. SOCO provides a systematic framework to balance an objective function with action switching. It thus lends itself well to video streaming, which needs to jointly optimize video quality, sustained playback, and bitrate smoothness.

Requirements. Driven by the above design gaps and opportunities, we identify three requirements that SODA should deliver. In particular, SODA should (i) achieve bitrate smoothness without

sacrificing video quality or sustained playback, (ii) be robust against volatile network conditions, and (iii) be easy to deploy in practice. Before delving into the details in the remainder of the paper, we provide a brief overview of how SODA satisfies these requirements:

- SODA leverages SOCO to *balance the trade-off* between video quality, sustained playback without rebuffers, and bitrate smoothness without frequent switches. Importantly, SODA focuses on steering the buffer level towards a target rather than weighing video quality against rebuffering duration (see Section 3.1).
- To achieve *robustness* against throughput variability, SODA is designed to satisfy the *exponentially decaying perturbation property*, which guarantees that SODA never operates too far away from the optimal trajectory in the face of prediction errors (see Section 4.2).
- To remain *computationally efficient*, SODA leverages an efficient approximate solver (see Section A.5 for proof and Algorithm 1 for implementation), that only requires evaluation of monotonic bitrate sequences (Section 4.3), which reduces the computational cost by two orders of magnitude over a brute-force solver.

3 SODA OVERVIEW

Given the design gaps, opportunities, and requirements, we set out to design a *theoretically sound* adaptive bitrate streaming (ABR) controller that minimizes bitrate switching without compromising video quality or increasing rebuffering time, thus providing a smooth viewing experience. To accomplish this, we deviate from the conventional segment-based ABR formulation and derive theoretical insights from a time-based ABR formulation. This enables us to incorporate throughput predictions into the controller in a principled way. Taking advantage of recent advancements in smoothed online convex optimization (SOCO), we can theoretically prove that SODA offers a near-optimal quality of experience (QoE) and is robust against throughput prediction errors.

3.1 A Time-Based ABR Formulation

Our time-based ABR formulation treats a video stream as a *continuous flow* rather than a discrete sequence of segments. Consider a streaming session that consists of N time intervals with fixed duration Δt in terms of *clock time* (not video time). The controller's task is to select a bitrate for each time interval from a set of available bitrates $\mathcal{R} \subset [r_{\min}, r_{\max}]$ to optimize for a combination of high quality, short rebuffering, and infrequent bitrate switching.

Let ω_n denote the average throughput during the n^{th} time interval, r_n the selected bitrate for that time interval, and x_n the buffer level immediately after that time interval. Our objective is to minimize the overall cost given as a linear combination of the three QoE components:

$$\sum_{n=1}^N \left(v(r_n) \cdot \frac{\omega_n \Delta t}{r_n} + \beta \cdot b(x_n) + \gamma \cdot c(r_n, r_{n-1}) \right), \quad (1)$$

where

- $v(r_n)$ is the **distortion cost**, which should be a positive, strictly decreasing, and convex function that models the encoding distortion, e.g., $v(r_n) = 1/r_n$. It is then weighted by the amount of video downloaded during that time interval, i.e., $\omega_n \Delta t / r_n$ because the controller downloads a *variable* amount of video during each fixed time interval.

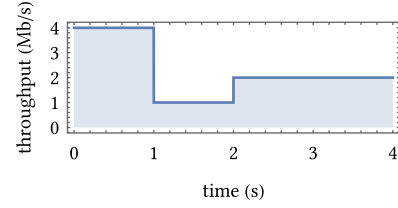


Figure 4: A sample throughput function used to illustrate why our time-based formulation is better for analysis.

- $b(x_n)$ is the **buffer cost**, which aims to stabilize the buffer level around a target level $\bar{x}$, i.e.,

$$b(x_n) = \begin{cases} (\bar{x} - x_n)^2 & x_n \leq \bar{x} \\ \epsilon(x_n - \bar{x})^2 & x_n > \bar{x} \end{cases},$$

where $\epsilon < 1$ is a small constant. Note that we purposely do not model the rebuffering time explicitly to avoid the pitfalls encountered by RobustMPC (Section 2) and as we show later, this helps SODA achieve theoretical performance guarantees (Section 4.2).

- $c(r_n, r_{n-1})$ is the **switching cost** from the previous bitrate to the current bitrate, e.g., $c(r_n, r_{n-1}) = (v(r_n) - v(r_{n-1}))^2$.

Coefficients β and γ are positive weights for the buffer and the switching cost respectively based on user preferences. The choices for the distortion and switching cost functions are flexible.

The *time-based* buffer dynamics are introduced into the optimization problem through the following constraint:

$$x_n = x_{n-1} + \frac{\omega_n \Delta t}{r_n} - \Delta t \in [0, x_{\max}],$$

where $\omega_n \Delta t / r_n$ accounts for the variable amount of video downloaded during a time interval and Δt accounts for the fixed amount of buffer drained during the same time interval. Note that we do not allow the controller to violate the buffer range constraint during the *optimization phase* when determining the bitrate. Of course, due to throughput prediction errors, this may sometimes be inevitable during the *execution phase* when applying the bitrate decision.

Why a Time-Based Formulation? The time-based formulation allows a cleaner theoretical analysis over a given throughput sequence $(\omega_1, \dots, \omega_N)$. For example, consider the throughput function shown in Figure 4. In the time-based formulation, we naturally have $\omega_1 = 4$, $\omega_2 = 1$, and $\omega_3 = \omega_4 = 2$ Mb/s given $\Delta t = 1$ s. By contrast, in the segment-based formulation, the throughput sequence becomes dependent on the bitrate sequence. Assuming the segment duration is also $L = 1$ s, if the controller chooses $r_1 = 2$ Mb/s and $r_2 = 2.5$ Mb/s, then it takes 0.5 and 1 s to download the first and second segments respectively, resulting in $\omega_1 = 4$ and $\omega_2 = 2.5$ Mb/s. As such, the segment based formulation gets causally biased due to bitrate selection $r_1, \dots, r_N$, which in turn makes it difficult to theoretically analyze the design [61].

Why Not Model Rebuffering Directly? Rebuffering is important to minimize from a user's perspective [7, 9]. However, in our optimization problem formulation, we did not explicitly model rebuffering like prior works [17, 46, 50]. Instead, we focus on stabilizing the buffer level around a target level with a smooth roll-off on both sides for the following reasons:

- Minimizing rebuffering directly is not theoretically tractable because it requires a binary penalty function that yields a non-zero penalty exactly when the buffer is empty. Instead, we employ a smoother penalty function that increases in magnitude when the buffer level falls below a desired target level. When there is a network issue, we start to penalize early when the buffer level decreases below the safe target level and we provide the largest penalty when the buffer level is empty. Using a smooth penalty function enables us to guarantee that SODA's optimization is strongly convex, which is key to our theoretical work. Our approach is analogous to the use of control barrier functions to ensure safety properties in control systems [30].
- Modeling rebuffering time directly makes the controller vulnerable to throughput prediction errors. Under a direct rebuffering objective, as long as the buffer level is above zero, there will be no penalty for the controller, even if the buffer level is dangerously close to 0. As a result, even small throughput prediction errors can lead to unexpected rebuffering.

3.2 Incorporating Throughput Predictions

In addition to facilitating theoretical analysis, our time-based formulation is crucial to ensuring the validity of throughput predictions over the prediction horizon. An important observation is that *bitrate decisions have no causal impact on how long the throughput predictions are valid for*. However, segment-based controllers such as MPC [17] and Fugu [46] intertwine throughput predictions and bitrate decisions in non-causal ways. In these designs, the throughput prediction horizon spans shorter periods of clock time when low bitrate is selected compared to when high bitrate is selected. In fact, their underlying assumption about the validity of the throughput prediction horizon can vary by $r_{\max}/r_{\min}$.

By contrast, the way we incorporate throughput predictions into SODA *does not* suffer from this issue. Specifically, just before each time interval, the controller is given access to a (not necessarily accurate) throughput prediction for the next K time intervals from a black-box throughput predictor. It is always assumed that the validity of the throughput prediction is $K\Delta t$, a fixed value. In general, a throughput predictor may output a different value for each of the next K time intervals, i.e., $\hat{\omega}_{n|n-1}, \hat{\omega}_{n+1|n-1}, \dots, \hat{\omega}_{n+K-1|n-1}$, where $\hat{\omega}_{m|n-1}$ ($m \geq n$) is the throughput prediction for the m^{th} time interval given previous download information up until the $(n-1)^{\text{th}}$ time interval. In other words, a throughput predictor can output a piecewise constant throughput function for the next $K\Delta t$ time. In practice, though, a typical throughput predictor outputs a single value that corresponds to a constant throughput function.

3.3 Control Mechanism

Inspired by the model predictive control framework, SODA selects a bitrate for each time interval by optimizing over the next K time intervals and then committing to the bitrate decision for the immediate next time interval, i.e., minimizing

$$\sum_{m=n}^{n+K-1} \left(v(r_m) \cdot \frac{\hat{\omega}_{m|n-1}\Delta t}{r_m} + \beta \cdot b(x_m) + \gamma \cdot c(r_m, r_{m-1}) \right) \quad (2a)$$

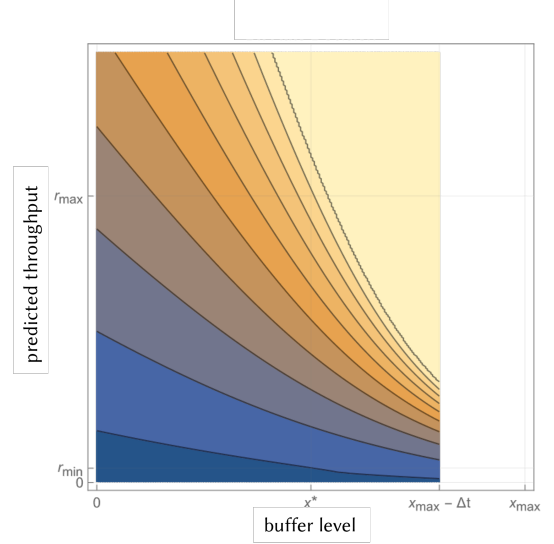


Figure 5: SODA's bitrate decision as a function of buffer level and predicted throughput. Dark blue to light orange represent low to high bitrate decisions. Notice that SODA becomes more aggressive in selecting higher bitrates as the buffer grows. The rightmost region is blank since SODA makes no downloads to prevent a buffer overflow.

$$\text{subject to } x_m = x_{m-1} + \frac{\hat{\omega}_{m|n-1}\Delta t}{r_m} - \Delta t, \quad (2b)$$

$$x_m \in [0, x_{\max}], \quad r_m \in \mathcal{R}, \quad (2c)$$

with respect to variables $r_n, \dots, r_{n+K-1}$ and then committing to only the first bitrate decision r_n . The behavior of SODA is visualized as a bitrate decision diagram in Figure 5 to provide readers with intuition about how SODA selects bitrates in practice.

As discussed in Section 2, solving this optimization problem is computationally expensive, furthermore, it is unclear what prediction horizon should be used and how accurate throughput predictions must be in order for SODA to perform well. We first analyze these questions theoretically (Section 4) and then present a practical implementation of SODA that answers these concerns (Section 5).

4 THEORETICAL DESIGN INSIGHTS

Our design of SODA is motivated by recent theoretical advances at the interface of learning and control [28, 38, 49, 54] and smoothed online convex optimization [25, 33, 55]. In particular, we design SODA to satisfy an *exponentially decaying perturbation property* that has been shown to ensure efficient and robust use of predictions in model predictive control policies [49, 56]. Intuitively, this property describes the behavior of the solution to the optimization problem defining SODA (Equation 2) as a function of problem parameters, including bandwidth predictions $\{\hat{\omega}_{m|n-1}\}_{n \leq m < n+K}$ and the previous buffer level/action pair (x_{n-1}, u_{n-1}) . Here, we define the *actions* as the inverse of the bitrates (i.e., $u_\tau = 1/r_\tau$ for all time step τ) and do a change of the variables to make the dynamics linear for the theoretical analysis. Under this property, when $\{\hat{\omega}_{m|n-1}\}_{n \leq m < n+K}$ are fixed, the optimal trajectory of (Equation 2) under the initial

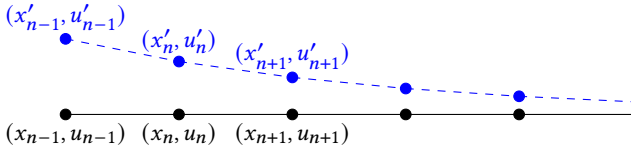


Figure 6: Illustration of the exponentially decaying perturbation property: When $\{\hat{\omega}_{m|n-1}\}_{n \leq m < n+K}$ are fixed, the optimal trajectories of Equation 2 under different initial buffer/action pairs converge exponentially toward each other.

buffer/action pair (x'_{n-1}, u'_{n-1}) converges exponentially toward the optimal trajectory under the pair (x_{n-1}, u_{n-1}) (see Figure 6 for an illustration). On the other hand, when the initial buffer/action pair is fixed, the impact of perturbing a prediction $\hat{\omega}_{m|n-1}$ on the first action u_n decays exponentially with respect to their temporal distance $(m - n)$. The formal definition of exponentially decaying perturbation generalizes the intuition above to consider the impact of perturbing any parameters on the entire optimal trajectory (see Definition A.1 in Appendix A).

Two metrics that we use to measure SODA's performance theoretically are *dynamic regret* and *competitive ratio*, which are standard in the literature of online optimization [25, 28, 33, 49, 54]. Specifically, let $\text{cost}(\text{ALG})$ denote the total cost incurred by an online algorithm ALG and $\text{cost}(\text{OPT})$ denote the offline optimal cost (Equation 1) an agent can incur if it has exact knowledge of all future bandwidth at the beginning. We say ALG achieves a dynamic regret of R if $\text{cost}(\text{ALG}) - \text{cost}(\text{OPT}) \leq R$ always holds, and ALG achieves a competitive ratio of C if $\text{cost}(\text{ALG}) \leq C \cdot \text{cost}(\text{OPT})$ always holds.

The key idea underlying our theoretical analysis is to leverage the exponential decay property to bound (i) the error that SODA incurs at every intermediate time step n due to its limited prediction power ($\hat{\omega}_{m|n-1} \neq \omega_m, K \ll N$), and (ii) the aggregation of such errors over the whole horizon N . Specifically, we define the notion of *per-step error* at a time step n as the distance between SODA's buffer/action pair and the optimal buffer/action pair that one could reach with exact predictions of all future bandwidths $\omega_n, \omega_{n+1}, \dots, \omega_N$ given the previous buffer/action pair (x_{n-1}, u_{n-1}) (Definition A.2). Using the principle of optimality, we reformulate the optimal buffer/action pair as an entry of the optimal trajectory from time n to $(n + K - 1)$ so that we can directly compare it with SODA's buffer/action pair under the exponentially decaying perturbation. Thus, we establish a bound on the per-step error that depends on the errors of predicting future bandwidths and the prediction horizon K (Lemma A.4). On the other hand, we also show that the aggregation of per-step errors does not grow linearly in time because the exponentially decaying perturbation guarantees that the impact of each previous per-step error vanishes exponentially over time (Lemma A.5). We present a proof outline and the detailed proofs in Appendix A. To prove the exponentially decaying perturbation, we require a technical assumption that guarantees the controller can “reach” any desired buffer level by choosing the largest/smallest bitrate (see Assumption A.1 in Appendix A for the formal statement). This assumption is used to eliminate extreme boundary cases in the

analysis, but we find SODA empirically performs very well even when this assumption is not strictly satisfied.

In this section, we set $\Delta t = 1$, $\mathcal{R} = [r_{\min}, r_{\max}]$, and $v(r) = 1/r$. Our results can apply to other distortion cost functions, e.g., $v(r) = \log(r_{\max}/r)$, as long as certain regularity conditions hold; see Appendix B for a discussion.

4.1 Exact Predictions

When the bandwidth predictions are accurate, a small prediction horizon is sufficient for SODA to achieve near-optimal performance. In practice, it is desirable to use a relatively small prediction horizon for a predictive controller like SODA because prediction errors grow dramatically as we predict further into the future. Fortunately, the exponential decay property that ensures good performance with only a few predictions. More formally, we present a theorem showing that a small prediction horizon is sufficient for SODA to achieve near-optimal performance when the predictions within this window are accurate (i.e., $\hat{\omega}_{m|n-1} = \omega_m$ for $m = n, \dots, n + K - 1$).

THEOREM 4.1. [Informal] *When the predictions of the bandwidth in future K steps are exact (i.e., $\hat{\omega}_{m|n-1} = \omega_m$ for $m = n, \dots, n + K - 1$) and the prediction horizon $K \geq O(1)$, SODA achieves a dynamic regret of $O(\rho^K N)$ and a competitive ratio of $1 + O(\rho^K)$, where $\rho < 1$ is the decay factor of the exponentially decaying perturbation property.*

The formal statement of Theorem 4.1 is given in Theorem A.3 in Appendix A. This result implies that SODA's performance approaches that of the optimal sequence of decisions *exponentially fast* in the prediction horizon size K ; thus, only a small prediction horizon length is necessary to obtain good performance.

4.2 Inexact Predictions

We now relax the exact prediction assumption to prove SODA's robustness to a certain level of prediction errors thanks to its exponentially decaying perturbation property.

THEOREM 4.2. [Informal] *Suppose the prediction error at each step is bounded above. The buffer level of SODA will never hit the constraint boundary, i.e., $0 < x_n < x_{\max}$. Further, define $\mathcal{E} = \rho^{2K} N + \sum_{\kappa=1}^K \rho^\kappa E_\kappa$, where E_κ is the total squared error for predicting κ steps into the future. SODA achieves a dynamic regret of $O(\sqrt{\mathcal{E}N} + \mathcal{E})$.*

The formal statement of Theorem 4.2 is given in Theorem A.8 in Appendix A. Theorem 4.2 shows that, if the buffer costs are “steep” and the prediction errors on the bandwidth are relatively small, SODA can achieve a sequence of buffer levels that stay safely away from the boundaries of buffer constraint $[0, x_{\max}]$. The dynamic regret of SODA depends on the magnitude of the prediction errors and the regret improves when the errors become smaller. SODA acquires this guarantee thanks to its maintenance of the buffer near a target level $\bar{x}$. In contrast, RobustMPC [17] doesn't offer the same performance guarantee, thus even small bandwidth prediction errors can cause the video to rebuffer if the buffer level is near zero.

4.3 Computational Efficiency

Solving the predictive optimization problem to determine the exact optimal solution can be unrealistic in the application of adaptive bitrate streaming, where each decision needs to be made in the

minimum possible time. A critical observation underlying the implementation of SODA is that it is sufficient to search only for bitrate sequences that are increasing or decreasing monotonically. We provide a theoretical justification in the following theorem.

THEOREM 4.3. [Informal] Suppose SODA is given the predictions that satisfy $\hat{\omega}_{n|n-1} = \dots = \hat{\omega}_{n+K-1|n-1}$ at an intermediate time step n . Then, the bitrate trajectory solved by SODA can be approximated by a feasible monotonic bitrate trajectory with an error of $O(K/\sqrt{\gamma})$.

The formal statement of Theorem 4.3 is given in Theorem A.9 in Appendix A. Theorem 4.3 shows that the true optimal solution becomes closer to monotonic as the weight γ of switching costs increases. While the theoretical bound can be conservative, we find that even with moderate γ , the (discrete) decision made under the monotonic heuristic is usually identical to the true optimal solution on a real trajectory (see Figure 8).

5 IMPLEMENTATION DETAILS

Given the theoretical design insights, we now discuss the practical implementation of the high-level design described in Section 3. There are three practical concerns that require discussion: (i) how to translate the time-based design to the segment-based schema; (ii) how to incorporate throughput predictions robustly; and (iii) how to solve the predictive optimization problem efficiently.

5.1 Segment-Based Schema

SODA is intrinsically a time-based controller, but in practice, a video must be downloaded segment by segment according to the MPEG-DASH standard. To reconcile with this requirement, we keep the optimization phase as is in the time-based format and empirically set Δt to be equal to the segment length. This choice is justified by the fact that in the steady state, the download time of a video segment is expected to be close to the segment length or much less than that [29]. To further minimize the likelihood of committing to a bitrate for significantly longer than Δt , we introduce another heuristic that the controller must select a bitrate no higher than $\min\{r \in \mathcal{R} : r \geq \hat{\omega}\}$.

5.2 Incorporating Predictions Robustly

According to Section 4.2, SODA is robust against prediction errors by design as long as there is no systematic bias in prediction errors. Given the diverse network conditions in the wild, we prefer simple throughput predictors which makes SODA highly deployable since there is no dependence on complex throughput predictors. In practice, we observe that prediction accuracy degrades as the prediction horizon increases (see Figure 7). Therefore, we limit the prediction horizon length to at most 10 s. This is also supported by our finding in Section 4.1 that a longer prediction horizon yields diminishing returns.

5.3 Efficient Approximate Solver

At SODA's core is the predictive optimization problem described in Section 3.3. Unfortunately, solving this problem on the fly is computationally challenging. One may propose enumerating all combinations of discretized throughputs, buffer levels, and previous bitrates in the form of an offline computed lookup table, as is the

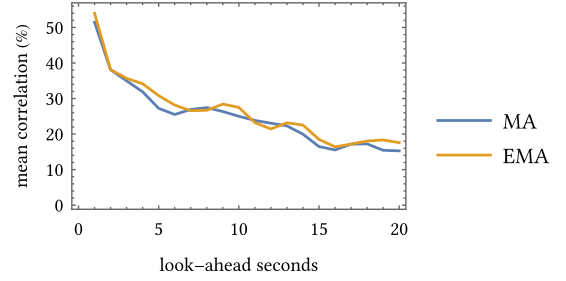


Figure 7: We profiled the performance of the two throughput predictors shipped with dash.js [64], i.e., moving average predictor and exponential moving average predictor. Both predictors have a high mean correlation (around 50%) in the immediate future but a very low mean correlation (around 15%) in the far future.

Algorithm 1: SODA's efficient approximate optimization solver. SearchDown is omitted for brevity due to symmetry. The current buffer level and the previous bitrate are denoted by x_0 and r_0 respectively.

```

function SEARCH( $\hat{\omega}, x_0, r_0, K$ )
    ( $r_{up}^*, obj_{up}^*$ )  $\leftarrow$  SEARCHUP( $\hat{\omega}, x_0, r_0, K$ )
    ( $r_{down}^*, obj_{down}^*$ )  $\leftarrow$  SEARCHDOWN( $\hat{\omega}, x_0, r_0, K$ )
    return  $r_{up}^* \neq \text{null} \wedge obj_{up}^* < obj_{down}^* ? r_{up}^* : r_{down}^*$ 

function SEARCHUP( $\hat{\omega}, x_0, r_0, K$ )
     $r_1^* \leftarrow \text{null}, obj^* \leftarrow \infty$ 
    foreach  $r_1 \in \{r \in \mathcal{R} : r > r_0\}$ 
         $x_1 \leftarrow x_0 + \hat{\omega}\Delta t/r_1 - \Delta t$ 
        if  $x_1 < 0$  then continue
         $obj \leftarrow v(r_1) \cdot \hat{\omega}\Delta t/r_1 + \beta \cdot b(x_1) + \gamma \cdot c(r_1, r_0)$ 
        if  $K > 1$  then
            ( $r_2^*, \Delta obj^*$ )  $\leftarrow$  SEARCHUP( $\hat{\omega}, x_1, r_1, K - 1$ )
            if  $r_2^* = \text{null}$  then continue
             $obj \leftarrow obj + \Delta obj^*$ 
        if  $obj < obj^*$  then  $r_1^* \leftarrow r_1, obj^* \leftarrow obj$ 
    return ( $r_1^*, obj^*$ )

```

case in FastMPC [17], however, this is neither flexible nor scalable in practice. A lookup table is specific to a particular set of bitrates, maximum player buffer, segment durations and byte sizes *etc.* thus needs to be recomputed when any of these quantities change. Furthermore, computing this lookup in live streaming is undesirable due to the additional computational and latency overhead it incurs. Instead, we opt for an efficient approximate solver.

SODA's approximate solver is designed to take advantage of the structure of the optimal solution presented in Section 4.3. Instead of searching through all possible bitrate sequences in the prediction horizon, the approximate solver only considers *monotonic* bitrate sequences, i.e., it imposes an additional constraint that $r_{n-1} \leq r_n \leq \dots \leq r_{n+K-1}$ or $r_{n-1} \geq r_n \geq \dots \geq r_{n+K-1}$. The pseudocode for a recursive implementation is shown in Algorithm 1.

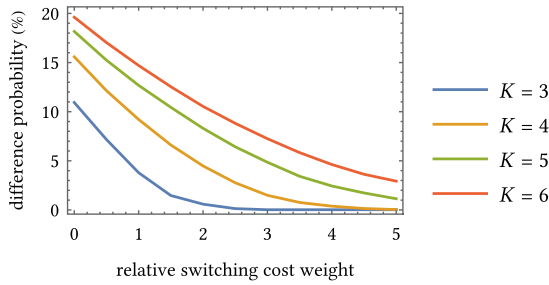


Figure 8: The probability that the bitrate decision produced by the approximate solver is different from that produced by the brute-force solver quickly converges to 0 as switching cost weight increases.

The approximate solver reduces the time complexity from $O(|\mathcal{R}|^K)$ (exponential in K) in the case of a brute-force search over all possible bitrate sequences in the prediction horizon down to $O\left(\binom{|\mathcal{R}|+K}{K}\right)$ (polynomial in K) and has a space complexity of $O(K)$ only. The time complexity can be further reduced by limiting extreme bitrate switches. In practice, SODA searches through at most around 200 bitrate sequences. According to our production deployment experience, the approximate solver did not impose a runtime burden even on low-end devices such as set-top boxes, which shows that SODA is highly practical.

Empirical results are shown in Figure 8 to validate the near-optimality of bitrate decisions produced by the approximate solver. For each algorithm configuration, we uniformly sample a million situations with different throughputs, buffer levels, and previous bitrates. Then, we count the probability that the bitrate decision produced by the approximate solver is different from that produced by the brute-force solver. The difference is negligible for a reasonable switching cost weight, e.g., below 5% for $K = 4$ and a relative switching cost weight of 2. Throughout the evaluation sections, we use this efficient implementation of SODA.

6 EVALUATION

To thoroughly evaluate SODA's performance, we conducted three levels of empirical evaluation: (i) large-scale numerical simulations, (ii) prototype evaluation in Puffer [46], and (iii) production deployment in Amazon Prime Video. This funnel approach allowed us to first systematically evaluate SODA against a variety of baselines in a wide range of controlled environments. Later, we narrowed the comparison target to a deployed and fine-tuned ABR controller in production using A/B tests on real user sessions.

Performance Metrics. To maintain consistency in terms of performance metrics with prior works such as [17, 22, 24, 46], a similar definition of QoE is adopted that consists of mean utility, rebuffering ratio, and switching rate. These correspond to the three main desired properties of adaptive bitrate streaming, i.e., high video quality, shorter rebuffering time, and less bitrate switching. All three QoE components are normalized between 0 and 1 for ease of interpretation. The precise definitions are as follows:

- **Mean Utility:** Unless otherwise noted, we use the commonly-used logarithmic utility function:

$$\bar{v} = \frac{1}{N} \sum_{i=1}^N \frac{\log(r_i/r_{\min})}{\log(r_{\max}/r_{\min})}.$$

- **Rebuffering Ratio:** The ratio of the total rebuffering time to the session duration, i.e., $\rho_{\text{rebuf}} = T_{\text{rebuf}}/T$.
- **Switching Rate:** Bitrate switch count divided by segment count minus one, i.e., $p_{\text{switch}} = N_{\text{switch}}/(N - 1)$.

The QoE score is simply a linear combination of the three QoE components, i.e., $\text{QoE} = \bar{v} - \beta \cdot \rho_{\text{rebuf}} - \gamma \cdot p_{\text{switch}}$. In this work, we chose $\beta = 10$ and $\gamma = 1$ to reflect the high importance of minimizing rebuffering time. To establish fair comparisons, we report the individual QoE components along with the QoE score.

6.1 Numerical Simulations

To perform large-scale numerical simulations, we implemented a highly optimized ABR simulator in C++ derived from Sabre [36]. The simulation accuracy of Sabre has been empirically validated against dash.js [64], the reference player for MPEG-DASH. We configured the simulator to allow a maximum buffer length of 20 seconds to replicate the typical live streaming conditions.

6.1.1 Experimental Setup. Our network dataset consists of about 38,000 hours of throughput traces compiled from the following three public sources:

- **Puffer Dataset** [46]: We downloaded and parsed all throughput traces from the Puffer platform during the time period of January 2023 to June 2023.
- **5G Dataset** [41]: A 5G network dataset from a major Irish mobile operator under both static and moving scenarios while downloading online content.
- **4G Dataset** [27]: A 4G network dataset from two major Irish mobile operators under both static and moving scenarios while downloading online content.

For all three datasets, we filtered out sessions shorter than 10 minutes and divided long sessions into consecutive 10-minute sessions, resulting in 230,322 sessions from the Puffer dataset, 88 sessions from the 5G dataset, and 187 sessions from the 4G dataset. Figure 9 illustrates the wide range of network conditions covered by these datasets. In general, the Puffer dataset represents better network conditions than the 5G and 4G datasets. The latter have much lower mean throughput and higher variance, thus posing a bigger challenge for ABR controllers.

To fully exercise our datasets, we considered a high-frame-rate 4K video encoded according to the YouTube recommended settings (1.5, 4, 7.5, 12, 24, and 60 Mb/s) [65] with a segment length of 2 seconds. For the 5G and 4G datasets, we considered the same video with the two highest bitrates removed. Finally, for throughput prediction, we opted for the exponential moving average (EMA) predictor, the default throughput predictor in dash.js.

6.1.2 Baseline ABR Controllers. We compared SODA against the following ABR controllers representative of each of the common ABR controller categories, i.e., throughput-based, buffer-based, and hybrid. They were tuned to our best efforts for our network datasets.

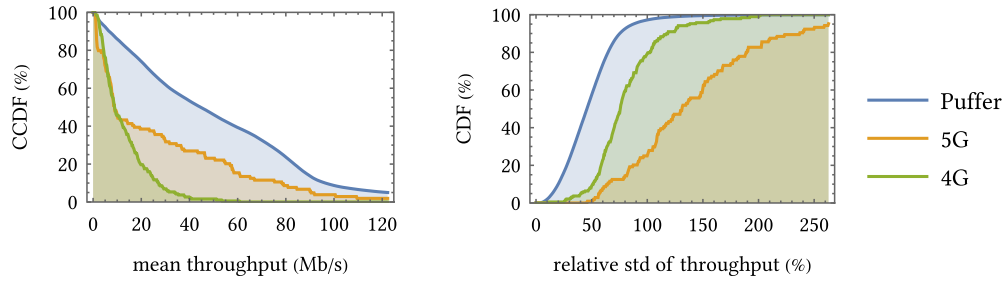


Figure 9: The mean throughput of the Puffer, 5G, and 4G datasets are 57.1, 31.3, and 13.0 Mb/s. The mean relative standard deviations of throughput of the Puffer, 5G, and 4G dataset are 47.2%, 133%, and 80.6%.

- **HYB** [24]: A heuristic throughput-based ABR controller that selects the highest bitrate without rebuffering.
- **BOLA** [36]: A buffer-based ABR controller derived from Lyapunov optimization. It provides theoretical guarantees about utility and rebuffering time only.
- **Dynamic** [44]: A production version of BOLA that dynamically switches between buffer mode and throughput mode in response to changes in network conditions. Additionally, it has low-buffer safety heuristic to reduce rebuffering and a switching avoidance heuristic to mitigate bitrate switching. It is the default ABR controller in `dash.js`.
- **MPC** [17]: One application of model predictive control to adaptive streaming that models utility, rebuffering time, and bitrate switching, without theoretical guarantees.

6.1.3 QoE Performance. The aggregate statistics for QoE scores and individual QoE components under each network dataset are shown in Figure 10. To better understand how the performance of different ABR controllers react to the intrinsic volatility of network conditions, we split the Puffer dataset into four quarters according to the relative standard deviation of throughput (Q1 represents the most stable network conditions, while Q4 represents the most volatile network conditions). In general, the more volatile network conditions are, the more the QoE performance of any ABR controller degrades, as evidenced by the trend in Figure 10 from left to right. Nonetheless, SODA consistently outperforms baseline ABR controllers under all network conditions. The improvement in terms of mean QoE scores compared to the best baseline across different network datasets ranges from 9.55% to 27.8%, which mainly stems from improvement in terms of smoothness (shorter rebuffering time and less bitrate switching). We discuss the improvement of SODA over each baseline ABR controller below:

- **SODA vs HYB.** HYB is not as robust as SODA under volatile network conditions. In addition, it switches up to 215% more since it does not consider bitrate switching.
- **SODA vs BOLA & Dynamic.** As mainly buffer-based ABR controllers, BOLA and Dynamic are fairly robust against volatile network conditions. Dynamic’s performance is what one would expect in a typical production environment. Nonetheless, SODA is able to achieve similar mean utilities without sacrificing mean rebuffering ratios, proving its outstanding robustness as a hybrid

ABR controller. Where SODA really shines though is its significantly lower mean switching rates. Despite Dynamic’s switching avoidance heuristic, SODA cuts down mean switching rates by as much as 70.4%, which demonstrates the superiority of theoretically-sound design.

- **SODA vs MPC.** MPC has high mean utilities and low mean switching rates under stable network conditions (see Puffer (Q1 variance) in Figure 10). However, the performance of MPC is tightly coupled with the intrinsic volatility of network conditions. Specifically, MPC suffers a lot in terms of mean rebuffering ratios especially under mobile network conditions. By contrast, SODA does not have this issue since it is robust against prediction errors by design, making it much more suitable for production deployment.

6.1.4 Intrinsic Sensitivity to Prediction Accuracy. In an effort to improve throughput prediction accuracy, several prior works have focused on designing more sophisticated throughput predictors such as C2SP [20], Fugu [46], and Xatu [50]. While these throughput predictors may offer higher prediction accuracy, they are complex and difficult to deploy, especially on compute or memory constrained devices [58]. In Section 4.2, we have showed that SODA is robust against prediction errors by design and does not require a sophisticated throughput predictor. We now demonstrate this empirically.

First, we replaced the throughput predictor used in simulations with a perfect short-term throughput predictor. Next, we gradually introduced more and more white noise to the perfect throughput predictions and observed how different ABR controllers behave accordingly. This experiment was conducted on a random subset of our network datasets with a size of 10,000 sessions. Note that throughput prediction discounts were turned off for all ABR controllers to reveal their *intrinsic* robustness.¹

The results are shown in Figure 11, from which we observe that all hybrid ABR controllers that take throughput predictions into account will inevitably be affected by prediction errors to some extent (BOLA is not affected since it is purely buffer-based). Nonetheless, SODA still consistently outperforms all baseline ABR controllers up to a noise level of 50%. For reference, EMA predictor has an empirical noise level of about 30% on the same sessions. More importantly, the QoE degradation of SODA is minimal up to the reference point of EMA predictor, i.e., about 10%, which reinforces

¹The ranking between different ABR controllers in this section may be different from that in Figure 10, which reveals that the robustness of certain ABR controllers should be attribute to throughput prediction discounts instead of intrinsic designs.

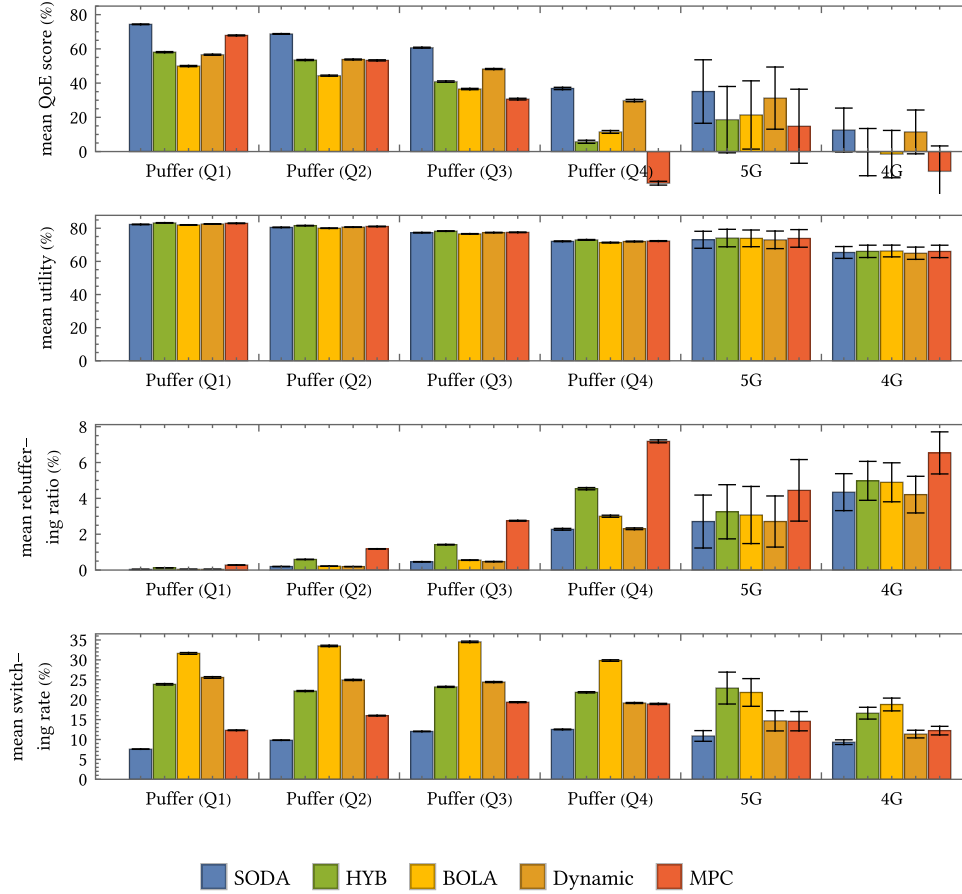


Figure 10: The mean QoE scores, utilities, rebuffering ratios and switching rates of SODA and baseline ABR controllers under each network dataset. The Puffer dataset is split into four quarters according the throughput variance (Q1 being lowest while Q4 being highest). SODA has consistently higher mean QoE scores and lower switching rates than all baseline ABR controllers under all network conditions. (Error bars represent 95% confidence intervals.)

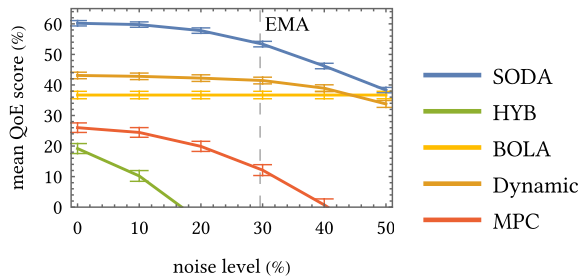


Figure 11: The mean QoE scores for SODA and baseline ABR controllers under variable amounts of white noise. (The error bars represent 95% confidence intervals.)

the idea that a practical deployment of SODA *does not* require a sophisticated throughput predictor.²

²In practice, we observe that EMA predictor is actually much better than a perfect short-term predictor with 30% white noise because the noise patterns are different, which means that real gap is less than 10%.

6.2 Prototype Evaluation

We next present emulation results from our local client-server deployment where we implemented SODA in the Puffer platform [46]. Thanks to Chrome DevTools' new capability to throttle WebSocket requests [59], we could replay our network datasets directly in Chrome using WebDriver [63]. The results are intended to highlight the robustness of different ABR controllers under actual browser-based playback. For these experiments, we allowed a maximum buffer length of 15 seconds, as set by Puffer.

6.2.1 Experimental Setup. The video source was a news clip encoded in five different resolutions (426×240 , 640×360 , 854×480 , 1280×720 , and 1920×1080) with a constant rate factor of 26 and a segment length of 2 seconds. To be fair to those learning-based ABR controllers trained specifically for the Puffer platform, we only considered the Puffer dataset. Since the average bitrate of the highest resolution is only about 2 Mb/s, we take a random subset of the Puffer dataset with a size of 1,000 sessions whose mean throughput is below 2 Mb/s to create challenging scenarios.

6.2.2 Baseline ABR Controllers. In response to the growing interest in learning-based throughput predictors and ABR controllers in the research community, we included two representative learning-based ABR controllers for local deployment on top of the major baseline ABR controllers from numerical simulations:

- **Fugu** [46]: Developed as part of the Puffer project, it features a learning-based stochastic throughput predictor, while its underlying control algorithm is similar to MPC.
- **CausalSimRL** [60]: A modern implementation of a reinforcement learning (RL)-based ABR controller Pensieve [22]. It is trained using CausalSim for the Puffer platform.

6.2.3 QoE Performance. Puffer employs structure similarity index measures (SSIM) [4] to quantify utility, thus to compare fairly using Puffer, we adapt mean utility to normalized mean SSIM, i.e., $\bar{v} = \text{SSIM} / \text{SSIM}_{\max}$. The definitions of rebuffering ratio, switching rate, and QoE score remain the same. The aggregate statistics or QoE scores and individual QoE components across all sessions are shown in Figure 12. SODA outperforms the best baseline (Fugu) by 30.4% in terms of mean QoE score. More importantly, SODA is the only ABR controller that achieves low mean rebuffering ratio and switching rate *simultaneously*, which translates to superior smoothness of adaptive streaming. We highlight comparisons with the new baseline ABR controllers below:

- **SODA vs MPC & Fugu.** MPC and Fugu are grouped together since, apart from the more sophisticated stochastic throughput predictor, Fugu shares a similar underlying control algorithm with MPC. While they both achieve slightly higher mean utilities than SODA and reasonably low mean switching rates, these benefits are overshadowed by worse mean rebuffering ratios (230% and 104% worse respectively). Although Fugu partially mitigates the rebuffering issue due to its stochastic throughput predictor, it is still not robust enough for challenging network conditions.
- **SODA vs CausalSimRL.** CausalSimRL achieves slightly higher mean utility than SODA and a reasonably low mean rebuffering ratio. However, it switches bitrates 86.3% more often than SODA. Due to the black-box nature of RL-based ABR controllers, it is hard to reason why this is the case. In addition, there exists no straightforward way to tune an RL-based controller in favor of one particular QoE component without a complete retraining. In a production environment, it is highly desirable that the trade-off between different QoE components is tunable.

6.3 Production Deployment

We now describe the results from deploying SODA for live streams delivered on Amazon Prime Video. The bitrate ladder for these video streams had the following bitrate rungs {0.2, 0.45, 0.8, 1.2, 1.8, 2, 4, 5, 6.5, 8.0} Mb/s. This range of available bitrates fully exercised SODA's bitrate adaptation capability as well as tested its runtime feasibility on actual devices. The experiment was run on three device families, including (i) desktops/laptops (HTML5 browsers), (ii) smart TVs, and (iii) set-top boxes. On all three platforms, SODA used a simple sliding window-based throughput predictor. All devices were 20 seconds behind live action, so they could accumulate at most 20 seconds of buffer. To compare performance with a production tuned baseline, we conducted large-scale A/B experiments

where customers were randomly assigned SODA or the production baseline controller. The experiment ran for more than 1 week with live streams delivered to more than 10 countries. In total, SODA sessions logged more than 50,000 streaming hours.

Figure 13 shows SODA's performance relative to the production deployed and tuned controller. First, notice that SODA consistently improves all the metrics across all device families, reducing the frequency of bitrate switching on set-top boxes by 88.8%. SODA really shines on HTML5 browsers where it reduced the mean rebuffering ratio by up to 53.0% in addition to 81.8% reduction in switching. This is because HTML5 browsers experience more volatility in network conditions compared to smart TVs and set-top boxes and thus present greater opportunity for improvement. Finally, notice that on all three platforms, the average duration of session increased, with 5.91% improvement on set-top boxes. Live streaming sessions for sports events routinely span multiple hours (e.g., 2-hour soccer broadcast, 3.5-hour cricket broadcast), so a 5.91% increase translates to more than 5 minutes duration.

Takeaways from Production Deployment. The production deployment shows that SODA is practical and can be widely deployed across different device types and network connections. Furthermore, to achieve its significant performance gains, it is sufficient for SODA to use simple sliding window-based throughput predictors.

7 RELATED WORK

7.1 Adaptive Bitrate Streaming

Bitrate adaptation has received significant attention from the multimedia research community. Buffer-based controllers like BBA [15] and BOLA [36, 44] make bitrate decisions based on buffer occupancy, while hybrid controllers like HYB [24], MPC [17] and DYNAMIC [44] combine throughput predictions with buffer occupancy to make decisions. SODA belongs to the latter category. There are also learning-based controllers such as Pensieve [22] that utilize reinforcement learning to learn a bitrate selection strategy. Another relevant stream of work focuses on improving the accuracy of throughput predictions, including CS2P [20], Fugu [46], and Xatu [50]. Our work makes no assumption on the quality of throughput predictions and does not require a sophisticated throughput predictor. Past works have also considered upgrading the downloaded segments through replacement [36] which we do not consider in this paper.

7.2 Video Quality of Experience

The advent of video content delivery networks [2, 6] in the late 1990's led to efforts in industry to define and measure quality metrics for video delivery. Since then the quality of video delivery is a well studied topic with early work on the Akamai Stream Analyzer system which defined metrics such as startup time, rebuffer ratio, bitrate and failures etc and measured these metrics using data derived from video players deployed around the world [3, 66]. Subsequently, [7] showed that a 1% increase in rebuffering correlated with a 3-minute reduction in the amount of time users streamed live content. A study on YouTube [21] found that bitrate fluctuations strongly correlate with a user abandoning the session. Beyond correlations, the first study [9] to establish a causal relationship between video quality and user behavior used quasi-experimental

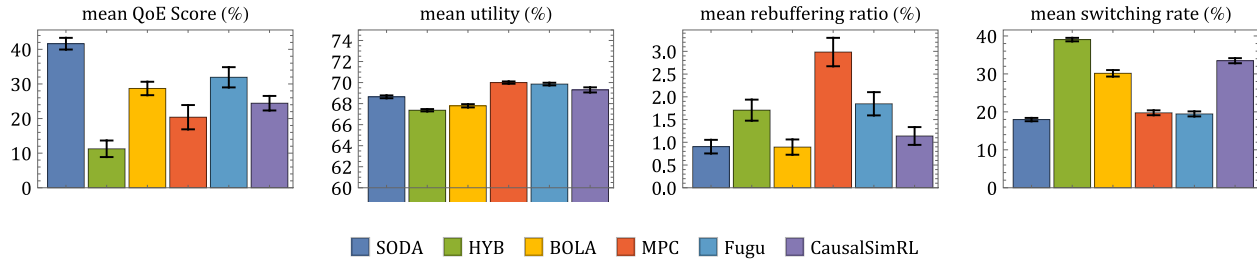


Figure 12: The mean QoE scores, utilities, rebuffering ratios, and switching rates from local deployment. SODA again has the highest mean QoE score and unlike all other baselines, *simultaneously* achieves ultra low mean rebuffering ratio and switching rate. (Error bars represent 95% confidence intervals.)

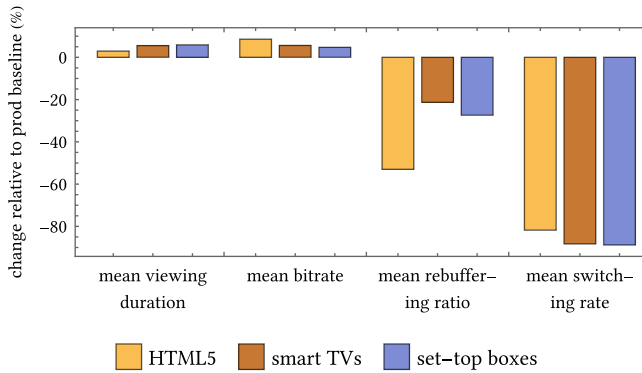


Figure 13: The change in mean viewing durations (higher is better), bitrates (higher is better), rebuffering ratios (lower is better), and switching rates (lower is better) of SODA compared to the production baseline.

designs (QEDs) to quantify the causal (adverse) impact of startup delay, rebuffering, and failures on user engagement, abandonment, and repeat viewership. A related work [11] built predictive models for user engagement based on QoE metrics. Our work leverages insights from these works in our ABR controller design.

7.3 Smoothed Online Convex Optimization

Our algorithm builds on recent developments in smoothed online convex optimization (SOCO), a variant of online optimization that penalizes switching between consecutive decisions via a “switching cost.” [25, 33, 34]. In recent years, the design and analysis of algorithms for SOCO has received considerable attention, e.g., [14, 25, 31, 32, 34, 37], with optimal online algorithms emerging in various settings [33, 42, 48, 62] and a variety of applications receiving attention [10, 12, 26, 43, 53, 55, 57]. SOCO’s switching cost model inspires our design of SODA for video streaming.

Our mathematical formulation of adaptive video streaming can be viewed as a specific example of online (optimal) control [54]. Similar to online optimization, online control seeks to design a controller to minimize the total cost incurred over a finite horizon. The theoretical bounds in this paper are most related to works that study how future predictions can improve online controller performance [39, 47, 49, 52]. Our proofs follow an analytic framework for

studying MPC-based algorithms via exponentially decaying perturbation bounds [49, 55, 56]. Our work shows that this decay property holds under our model of adaptive video streaming, allowing us to establish performance guarantees for SODA.

8 LIMITATIONS AND FUTURE WORK

An emerging genre (but, still a small fraction) of live streaming is ultra-low latency live streams where the delay between the capture of an event and its display to the user is required to be of the order of a few seconds, as opposed to 10 to 20 seconds for the traditional live streams used in our current work. In future work, we would like to study if our SOCO-based strategy can be adapted for ultra-low latency live streams with buffer lengths in the order of a few seconds. The main challenge with ultra-small buffer sizes is that it is harder to prevent rebuffering and bitrate switching in this regime as the ABR controller needs to react to network fluctuations in a significantly shorter amount of time.

9 CONCLUSION

In this work, we propose a smoothness-optimized dynamic adaptive (SODA) controller that addresses this issue in a theoretically sound way. Thanks to SODA’s robustness against prediction errors and low runtime complexity, it is readily deployable in a wide range of production environments. Through numerical simulations and prototype evaluation, we show that SODA consistently outperforms the state-of-the-art baselines. More importantly, we deployed SODA in a major video streaming provider where SODA significantly reduced bitrate switching by up to 88.8% compared to a fine-tuned production baseline. SODA’s novel time-based ABR formulation and theoretical insights shed new light on how to achieve consistent high-quality video streaming.

ACKNOWLEDGMENTS

We thank the anonymous reviewers and our shepherd for their valuable feedback, as well as colleagues at Amazon Prime Video for their support with the production deployment of SODA. This work was funded by NSF under grants CAREER-204564, CCF-2325956, CNS-1763617, CNS-1901137, CNS-2102963, CNS-2106299, CNS-2106403, CNS-2106463, CNS-2146814, CPS-2136197, and NGSDI-2105648, as well as an Amazon Research Award. The research of Yiheng Lin was additionally supported by Amazon AI4Science Fellowship and PIMCO Graduate Fellowship in Data Science.

REFERENCES

- [1] Mun Choon Chan and Ramachandran Ramjee. 2002. TCP/IP Performance over 3G Wireless Links with Rate and Delay Variation. In *Proceedings of the 8th Annual International Conference on Mobile Computing and Networking (MobiCom '02)*. Atlanta, Georgia, USA, 71–82. ISBN: 158113486X. doi: 10.1145/570645.570655.
- [2] John Dilley, Bruce M. Maggs, Jay Parikh, Harald Prokop, Ramesh K. Sitaraman, and William E. Weihl. 2002. Globally Distributed Content Delivery. *IEEE Internet Computing*, 6, 5, 50–58.
- [3] R.K. Sitaraman and R.W. Barton. 2003. Method and apparatus for measuring stream availability, quality and performance. US Patent 7,010,598. (Feb. 2003).
- [4] Z. Wang, A.C. Bovik, H.R. Sheikh, and E.P. Simoncelli. 2004. Image Quality Assessment: From Error Visibility to Structural Similarity. *IEEE Transactions on Image Processing*, 13, (Apr. 2004), 600–612, 4, (Apr. 2004). doi: 10.1109/TIP.2003.819861.
- [5] Junxian Huang, Qiang Xu, Birjodh Tiwana, Z. Morley Mao, Ming Zhang, and Paramvir Bahl. 2010. Anatomizing Application Performance Differences on Smartphones. In *Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services (MobiSys '10)*. San Francisco, California, USA, 165–178. ISBN: 9781605589855. doi: 10.1145/1814433.1814452.
- [6] E. Nygren, Ramesh K. Sitaraman, and J. Sun. 2010. The Akamai Network: A platform for high-performance Internet applications. *ACM SIGOPS Operating Systems Review*, 44, 3, 2–19.
- [7] Florin Dobrian, Vyas Sekar, Asad Awan, Ion Stoica, Dilip Joseph, Aditya Ganjam, Jibin Zhan, and Hui Zhang. 2011. Understanding the Impact of Video Quality on User Engagement. In *Proceedings of the ACM SIGCOMM 2011 Conference (SIGCOMM '11)*. Toronto, Ontario, Canada, 362–373. ISBN: 9781450307970. doi: 10.1145/2018436.2018478.
- [8] Te-Yuan Huang, Nikhil Handigol, Brandon Heller, Nick McKeown, and Ramesh Johari. 2012. Confused, timid, and unstable. In *Proceedings of the 2012 Internet Measurement Conference*. ACM, New York, NY, USA, (Nov. 2012), 225–238. ISBN: 9781450317054. doi: 10.1145/2398776.2398800.
- [9] S. Shunmuga Krishnan and Ramesh K. Sitaraman. 2012. Video Stream Quality Impacts Viewer Behavior: Inferring Causality Using Quasi-Experimental Designs. In *Proceedings of the 2012 Internet Measurement Conference (IMC '12)*. Boston, Massachusetts, USA, 211–224. ISBN: 9781450317054. doi: 10.1145/2398776.2398799.
- [10] Minghong Lin, Zhenhua Liu, Adam Wierman, and Lachlan LH Andrew. 2012. Online algorithms for geographical load balancing. In *Proceedings of the International Green Computing Conference (IGCC)*, 1–10.
- [11] Athula Balachandran, Vyas Sekar, Aditya Akella, Srinivasan Seshan, Ion Stoica, and Hui Zhang. 2013. Developing a predictive model of quality of experience for internet video. *SIGCOMM Comput. Commun. Rev.*, 43, 4, (Aug. 2013), 339–350. doi: 10.1145/2534169.2486025.
- [12] Minghong Lin, Adam Wierman, Lachlan L. H. Andrew, and Eno Thereska. 2013. Dynamic Right-Sizing for Power-Proportional Data Centers. *IEEE/ACM Transactions on Networking*, 21, 5, (Oct. 2013), 1378–1391. doi: 10.1109/TNET.2012.2226216.
- [13] Masoud Badii, Na Li, and Adam Wierman. 2015. Online convex optimization with ramp constraints. In *2015 54th IEEE Conference on Decision and Control (CDC)*. IEEE, 6730–6736.
- [14] Nikhil Bansal, Anupam Gupta, Ravishankar Krishnaswamy, Kirk Pruhs, Kevin Schewior, and Cliff Stein. 2015. A 2-Competitive Algorithm For Online Convex Optimization With Switching Costs. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2015)* (Leibniz International Proceedings in Informatics (LIPIcs)). Naveen Garg, Klaus Jansen, Anup Rao, and José D. P. Rolim, (Eds.) Vol. 40. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 96–109. doi: 10.4230/LIPIcs.APPROX-RANDOM.2015.96.
- [15] Te-Yuan Huang, Ramesh Johari, Nick McKeown, Matthew Trunnell, and Mark Watson. 2015. A Buffer-Based Approach to Rate Adaptation: Evidence from a Large Video Streaming Service. *ACM SIGCOMM Computer Communication Review*, 44, (Feb. 2015), 187–198, 4, (Feb. 2015). doi: 10.1145/2740070.2626296.
- [16] Yan Liu and Jack Y. B. Lee. 2015. An Empirical Study of Throughput Prediction in Mobile Data Networks. In *2015 IEEE Global Communications Conference (GLOBECOM)*, 1–6. doi: 10.1109/GLOCOM.2015.7417858.
- [17] Xiaoqi Yin, Abhishek Jindal, Vyas Sekar, and Bruno Sinopoli. 2015. A Control-Theoretic Approach for Dynamic Adaptive Video Streaming over HTTP. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. ACM, New York, NY, USA, (Aug. 2015), 325–338. ISBN: 9781450335423. doi: 10.1145/2785956.2787486.
- [18] Mojgan Ghasemi, Partha Kanuparth, Ahmed Mansy, Theophilus Benson, and Jennifer Rexford. 2016. Performance Characterization of a Commercial Video Streaming Service. In *Proceedings of the 2016 Internet Measurement Conference*. ACM, New York, NY, USA, (Nov. 2016), 499–511. ISBN: 9781450345262. doi: 10.1145/2987443.2987481.
- [19] he1enh. 2016. Reproducing Network Research. (May 2016). <https://reproducinetworkresearch.wordpress.com/2016/05/30/cs244-16-failed-experiments-with-fastmpc-integrating-rate-based-adaptive-streaming-into-vlc/>.
- [20] Yi Sun, Xiaoqi Yin, Junchen Jiang, Vyas Sekar, Fuyuan Lin, Nanshu Wang, Tao Liu, and Bruno Sinopoli. 2016. CS2P: Improving Video Bitrate Selection and Adaptation with Data-Driven Throughput Prediction. In *Proceedings of the 2016 ACM SIGCOMM Conference*. ACM, New York, NY, USA, (Aug. 2016), 272–285. ISBN: 9781450341936. doi: 10.1145/2934872.2934898.
- [21] Christos George Bampis, Zhi Li, Anush Krishna Moorthy, Ioannis Katsavounidis, Anne Aaron, and Alan Conrad Bovik. 2017. Study of Temporal Effects on Subjective Video Quality of Experience. *IEEE Transactions on Image Processing*, 26, 11, 5217–5231. doi: 10.1109/TIP.2017.2729891.
- [22] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. 2017. Neural Adaptive Video Streaming with Pensieve. In ACM, (Aug. 2017), 197–210. ISBN: 9781450346535. doi: 10.1145/3098822.3098843.
- [23] Yanyuan Qin, Ruofan Jin, Shuai Hao, Krishna R. Pattipati, Feng Qian, Subhabrata Sen, Bing Wang, and Chaogun Yue. 2017. A control theoretic approach to ABR video streaming: A fresh look at PID-based rate adaptation. In *IEEE INFOCOM 2017 - IEEE Conference on Computer Communications*. IEEE, (May 2017), 1–9. ISBN: 978-1-5090-5336-0. doi: 10.1109/INFOCOM.2017.8057056.
- [24] Zahaib Akhtar, Yun Seong Nam, Ramesh Govindan, Sanjay Rao, Jessica Chen, Ethan Katz-Bassett, Bruno Ribeiro, Jibin Zhan, and Hui Zhang. 2018. Oboe: Auto-Tuning Video ABR Algorithms to Network Conditions. In ACM, (Aug. 2018), 44–58. ISBN: 9781450355674. doi: 10.1145/3230543.3230558.
- [25] Niangujun Chen, Gautam Goel, and Adam Wierman. 2018. Smoothed Online Convex Optimization in High Dimensions via Online Balanced Descent. In *Proceedings of Conference On Learning Theory (COLT)*, 1574–1594.
- [26] Yingying Li, Guannan Qu, and Na Li. 2018. Online Optimization with Predictions and Switching Costs: Fast Algorithms and the Fundamental Limit. (2018). arXiv: 1801.07780v3 [math.OC].
- [27] Darijo Raca, Jason J. Quinlan, Ahmed H. Zahran, and Cormac J. Sreenan. 2018. Beyond Throughput: A 4G LTE Dataset with Channel and Context Metrics. In *Proceedings of the 9th ACM Multimedia Systems Conference*. ACM, New York, NY, USA, (June 2018), 460–465. ISBN: 9781450351928. doi: 10.1145/3204949.3208123.
- [28] Naman Agarwal, Brian Bullins, Elad Hazan, Sham Kakade, and Karan Singh. 2019. Online control with adversarial disturbances. In *International Conference on Machine Learning*. PMLR, 111–119.
- [29] Zahaib Akhtar, Yaguang Li, Ramesh Govindan, Emir Halepovic, Shuai Hao, Yan Liu, and Subhabrata Sen. 2019. AVIC: A Cache for Adaptive Bitrate Video. In *Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies (CoNEXT '19)*. Association for Computing Machinery, Orlando, Florida, 305–317. ISBN: 9781450369985. doi: 10.1145/3359989.3365423.
- [30] Aaron D Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. 2019. Control barrier functions: theory and applications. In *2019 18th European control conference (ECC)*. IEEE, 3420–3431.
- [31] C.J. Argue, Sébastien Bubeck, Michael B. Cohen, Anupam Gupta, and Yin Tat Lee. 2019. A Nearly-Linear Bound for Chasing Nested Convex Bodies. In *Proceedings of the 2019 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. Proceedings. Society for Industrial and Applied Mathematics, (Jan. 2019), 117–122. doi: 10.1137/1.9781611975482.8.
- [32] Sébastien Bubeck, Bo'az Klartag, Yin Tat Lee, Yuanzhi Li, and Mark Sellke. 2019. Chasing Nested Convex Bodies Nearly Optimally. In *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms (SODA)*. Proceedings. Society for Industrial and Applied Mathematics, (Dec. 2019), 1496–1508. doi: 10.1137/1.9781611975994.91.
- [33] Gautam Goel, Yiheng Lin, Haoyuan Sun, and Adam Wierman. 2019. Beyond online balanced descent: An optimal algorithm for smoothed online optimization. *Advances in Neural Information Processing Systems*, 32.
- [34] Gautam Goel and Adam Wierman. 2019. An Online Algorithm for Smoothed Regression and LQR Control. In *Proceedings of the Machine Learning Research*. Vol. 89, 2504–2513. <http://proceedings.mlr.press/v89/goel19a.html>.
- [35] Ming Shi, Xiaojun Lin, and Lei Jiao. 2019. On the value of look-ahead in competitive online convex optimization. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 3, 2, 22.
- [36] Kevin Spiteri, Ramesh Sitaraman, and Daniel Sparacio. 2019. From Theory to Practice: Improving Bitrate Adaptation in the DASH Reference Player. *ACM Transactions on Multimedia Computing, Communications, and Applications*, 15, 2s, (Apr. 2019), 1–29. doi: 10.1145/3336497.
- [37] C. J. Argue, Anupam Gupta, and Guru Guruganesh. 2020. Dimension-Free Bounds for Chasing Convex Functions. In *Proceedings of Thirty Third Conference on Learning Theory*. PMLR, (July 2020), 219–241. Retrieved Feb. 4, 2022 from.
- [38] Sarah Dean, Horia Mania, Nikolai Matni, Benjamin Recht, and Stephen Tu. 2020. On the sample complexity of the linear quadratic regulator. *Foundations of Computational Mathematics*, 20, 4, 633–679.
- [39] Yingying Li, Guannan Qu, and Na Li. 2020. Online optimization with predictions and switching costs: Fast algorithms and the fundamental limit. *IEEE Transactions on Automatic Control*, 66, 10, 4761–4768.